

OPERATING SYSTEMS (CS304)

Banker's Algorithm Example

Learning Objectives

Understand Banker's Algorithm, safe state, unsafe state, resource allocation, and solve numerical examples step-by-step.

Introduction

Banker's Algorithm is a deadlock avoidance algorithm proposed by Edsger Dijkstra. It determines whether granting a resource request will leave the system in a safe state.

What is a Safe State?

A system is in a safe state if there exists at least one sequence of process execution such that every process can complete without causing deadlock.

What is an Unsafe State?

An unsafe state is a condition where the system cannot guarantee that all processes can complete successfully.

Data Structures Used

- Available Matrix
- Allocation Matrix
- Maximum Matrix
- Need Matrix

Need = Maximum - Allocation

Algorithm Steps

1. Calculate Need Matrix.
2. Find a process whose Need is less than Available resources.
3. Allocate resources and execute process.
4. Release resources after completion.
5. Repeat until all processes finish.

Worked Example

Available Resources: A = 3, B = 3, C = 2

Allocation Matrix: P0 = (0,1,0)

P1 = (2,0,0)

P2 = (3,0,2)

Maximum Matrix: P0 = (7,5,3)

P1 = (3,2,2)

P2 = (9,0,2)

Need Matrix = Maximum - Allocation

Safe Sequence

After evaluating Need and Available resources, one possible safe sequence is: $P1 \rightarrow P0 \rightarrow P2$
Since all processes complete successfully, the system is in a safe state.

Advantages of Banker's Algorithm

- Prevents deadlocks
- Efficient resource utilization
- Determines safe states
- Supports resource allocation decisions

Limitations

- Requires advance knowledge of maximum resource needs
- Computational overhead
- Not suitable for highly dynamic systems

Exam Important Questions

1. Define Banker's Algorithm.
2. What is a Safe State?
3. Differentiate Safe and Unsafe States.
4. Explain the Need Matrix.
5. Solve a numerical problem using Banker's Algorithm.

Summary

Banker's Algorithm is an important deadlock avoidance technique that ensures resource allocation decisions maintain system safety and prevent deadlock conditions.